

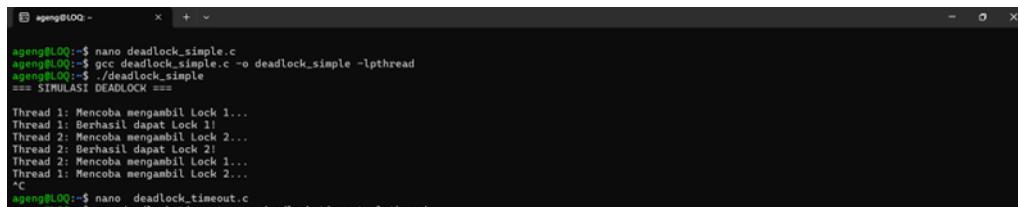
Tugas Praktikum 6

Tugas 1: Analisis Deadlock

Jalankan program `deadlock_simple.c` dan jelaskan:

1. Mengapa program mengalami deadlock?
2. Kondisi deadlock apa saja yang terpenuhi?
3. Gambar diagram resource allocation graph-nya

Jawaban:



```
ageng@LQ:~$ nano deadlock_simple.c
ageng@LQ:~$ gcc deadlock_simple.c -o deadlock_simple -lpthread
ageng@LQ:~$ ./deadlock_simple
== SIMULASI DEADLOCK ==
Thread 1: Mencoba mengambil Lock 1...
Thread 1: Berhasil dapat lock 1!
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Berhasil dapat Lock 2!
Thread 2: Mencoba mengambil Lock 1...
Thread 1: Mencoba mengambil Lock 2...
^C
ageng@LQ:~$ nano deadlock_timeout.c
```

1. Mengapa program mengalami deadlock?

Program ini mengalami deadlock karena dua thread saling menunggu resource (lock) yang dimiliki oleh thread lain dan tidak pernah dilepas.

Berikut alurnya:

1. Thread 1 mengunci lock1.
2. Thread 2 mengunci lock2.
3. Thread 1 kemudian ingin mengunci lock2, tapi lock2 sudah dipegang oleh Thread 2, jadi Thread 1 harus menunggu.
4. Thread 2 kemudian ingin mengunci lock1, tapi lock1 sudah dipegang oleh Thread 1, jadi Thread 2 juga menunggu.
5. Akibatnya, kedua thread menunggu selamanya — tidak ada yang bisa melanjutkan eksekusi untuk melepaskan lock.

Kondisi ini disebut deadlock karena tidak ada kemajuan yang mungkin terjadi.

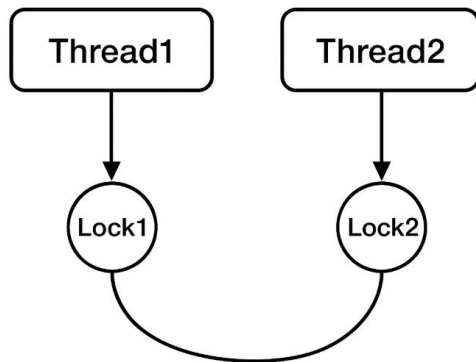
Program `deadlock_simple.c` mengalami kondisi deadlock karena dua thread saling menunggu resource yang dipegang oleh thread lain. Berikut analisis empat kondisi Coffman yang menyebabkan deadlock.

No	Kondisi Deadlock (Coffman)	Penjelasan dalam Program	Terpenuhi?
1	Mutual Exclusion (Saling Menyendiri)	Setiap mutex lock (lock1, lock2) hanya bisa dipegang oleh satu thread pada satu waktu. Thread lain harus menunggu.	Ya
2	Hold and Wait (Menahan dan Menunggu)	Thread 1 memegang lock1 dan menunggu lock2; Thread 2 memegang lock2 dan menunggu lock1.	Ya
3	No Preemption (Tidak Ada Pengambilalihan)	Mutex tidak bisa direbut secara paksa; hanya thread pemilik yang bisa melepaskannya.	Ya
4	Circular Wait (Menunggu Secara Melingkar)	Thread 1 → menunggu Lock2 (dipegang Thread2) → menunggu Lock1 (dipegang Thread1) — membentuk siklus tunggu.	Ya

Kesimpulan

Keempat kondisi deadlock terpenuhi, yaitu Mutual Exclusion, Hold and Wait, No Preemption, dan Circular Wait. Akibatnya program mengalami deadlock karena kedua thread saling menunggu tanpa ada yang bisa melanjutkan eksekusi.

Gambar diagram resource allocation graph-nya



Penjelasan:

- Panah dari thread ke resource berarti thread meminta resource.
 - Thread1 → Lock2 (Thread 1 menunggu Lock 2)
 - Thread2 → Lock1 (Thread 2 menunggu Lock 1)
- Panah dari resource ke thread berarti resource dimiliki oleh thread itu.
 - Lock1 → Thread1 (Lock1 dipegang Thread 1)
 - Lock2 → Thread2 (Lock2 dipegang Thread 2)

Lingkaran antara Thread1 ↔ Lock1 ↔ Thread2 ↔ Lock2 ↔ Thread1 membentuk siklus circular wait, tanda khas deadlock.

Tugas 2: Implementasi Pencegahan

Modifikasi program deadlock_simple.c menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

Tugas 2: Implementasi Pencegahan Deadlock

Tujuan:

Mencegah deadlock pada program deadlock_simple.c menggunakan dua metode:

1. Lock Ordering (pengurutan pengambilan kunci)
2. Try-Lock dengan Retry (pengambilan kunci dengan percobaan ulang)

Metode Lock Ordering

Prinsip:

Pastikan semua thread mengambil kunci dalam urutan yang sama (misalnya, selalu lock1 dulu, baru lock2).

Dengan begitu, tidak akan ada siklus tunggu (circular wait).

1. Lock ordering

Kode: deadlock_lock_ordering.c

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
// Deklarasi dua mutex
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
// Fungsi untuk Thread 1
```

```
void* thread1_func(void* arg) {  
  
    printf("Thread 1: Mencoba Lock 1...\n");  
  
    pthread_mutex_lock(&lock1);  
  
    printf("Thread 1: Berhasil dapat Lock 1!\n");  
  
    sleep(1);  
  
    printf("Thread 1: Mencoba Lock 2...\n");  
  
    pthread_mutex_lock(&lock2);  
  
    printf("Thread 1: Berhasil dapat Lock 2!\n");  
  
  
    printf("Thread 1: Selesai bekerja\n");  
  
    pthread_mutex_unlock(&lock2);  
  
    pthread_mutex_unlock(&lock1);  
  
    return NULL;  
}
```

```
// Fungsi untuk Thread 2
```

```
void* thread2_func(void* arg) {  
  
    printf("Thread 2: Mencoba Lock 1...\n");  
  
    pthread_mutex_lock(&lock1);  
  
    printf("Thread 2: Berhasil dapat Lock 1!\n");  
  
    sleep(1);
```

```

printf("Thread 2: Mencoba Lock 2...\n");

pthread_mutex_lock(&lock2);

printf("Thread 2: Berhasil dapat Lock 2!\n");


printf("Thread 2: Selesai bekerja\n");

pthread_mutex_unlock(&lock2);

pthread_mutex_unlock(&lock1);

return NULL;
}


int main() {

pthread_t t1, t2;

printf("=== Pencegahan Deadlock: Lock Ordering ===\n\n");

pthread_create(&t1, NULL, thread1_func, NULL);

pthread_create(&t2, NULL, thread2_func, NULL);


pthread_join(t1, NULL);

pthread_join(t2, NULL);


printf("\nProgram selesai tanpa deadlock.\n");

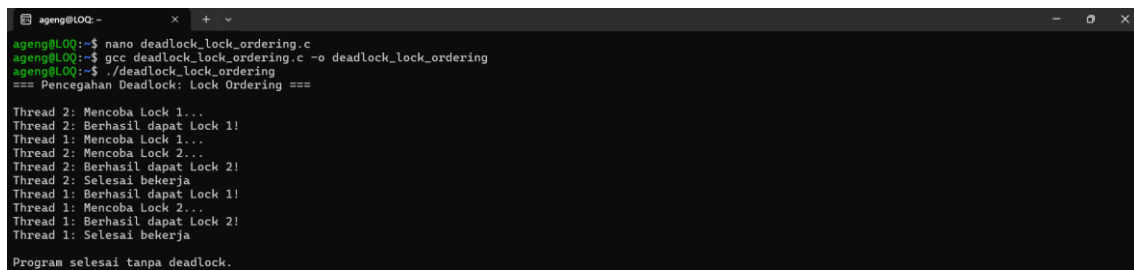
return 0;

}

```

Hasil:

Program tidak akan deadlock, karena kedua thread selalu mengambil lock dalam urutan yang sama: lock1 → lock2.



```
ageng@LOQ: ~$ nano deadlock_lock_ordering.c
ageng@LOQ: ~$ gcc deadlock_lock_ordering.c -o deadlock_lock_ordering
ageng@LOQ: ~$ ./deadlock_lock_ordering
=== Pencegahan Deadlock: Lock Ordering ===
Thread 2: Mencoba Lock 1...
Thread 2: Berhasil dapat Lock 1!
Thread 1: Mencoba Lock 1...
Thread 2: Mencoba Lock 2...
Thread 2: Berhasil dapat Lock 2!
Thread 2: Selesai bekerja
Thread 1: Berhasil dapat Lock 1!
Thread 1: Mencoba Lock 2...
Thread 1: Berhasil dapat Lock 2!
Thread 1: Selesai bekerja
Program selesai tanpa deadlock.
```

Metode Try-Lock dengan Retry

Prinsip:

Gunakan `pthread_mutex_trylock()` untuk mencoba mengunci resource.

Jika gagal (karena sedang dipegang thread lain), thread akan melepaskan lock yang sudah dimiliki dan mencoba lagi setelah jeda.

2. Try-lock dengan retry

Kode: `deadlock_trylock_retry.c`

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* thread_func(void* arg) {
```

```
    int id = *(int*)arg;
```

```
    int got_lock1, got_lock2;
```

```

while (1) {

    got_lock1 = pthread_mutex_trylock(&lock1);

    if (got_lock1 == 0) {

        printf("Thread %d: Dapat Lock 1\n", id);

        sleep(1); // simulasi kerja


        got_lock2 = pthread_mutex_trylock(&lock2);

        if (got_lock2 == 0) {

            printf("Thread %d: Dapat Lock 2\n", id);

            printf("Thread %d: Selesai bekerja\n", id);


            pthread_mutex_unlock(&lock2);

            pthread_mutex_unlock(&lock1);

            break; // keluar loop setelah sukses

        } else {

            // gagal dapat lock2 → lepaskan lock1 dan coba lagi

            pthread_mutex_unlock(&lock1);

            printf("Thread %d: Gagal dapat Lock 2, retry...\n", id);

            sleep(1);

        }

    } else {

        printf("Thread %d: Gagal dapat Lock 1, retry...\n", id);

        sleep(1);
    }
}

```



```

    }

}

return NULL;

}

int main() {

    pthread_t t1, t2;

    int id1 = 1, id2 = 2;

    printf("=== Pencegahan Deadlock: Try-Lock dengan Retry ===\n\n");

    pthread_create(&t1, NULL, thread_func, &id1);

    pthread_create(&t2, NULL, thread_func, &id2);

    pthread_join(t1, NULL);

    pthread_join(t2, NULL);

    printf("\nProgram selesai tanpa deadlock.\n");

    return 0;

}

```

Hasil:

Tidak ada deadlock, karena thread yang gagal mendapat lock akan mundur (retry) dan memberi kesempatan thread lain untuk melanjutkan.

```
Program selesai tanpa deadlock.
ageng@LOQ:~$ nano deadlock_trylock_retry.c
ageng@LOQ:~$ gcc deadlock_trylock_retry.c -o deadlock_trylock_retry
ageng@LOQ:~$ ./deadlock_trylock_retry
=== Pencegahan Deadlock: Try-Lock dengan Retry ===

Thread 1: Dapat Lock 1
Thread 2: Gagal dapat Lock 1, retry...
Thread 1: Dapat Lock 2
Thread 1: Selesai bekerja
Thread 2: Dapat Lock 1
Thread 2: Dapat Lock 2
Thread 2: Selesai bekerja

Program selesai tanpa deadlock.
ageng@LOQ:~$ |
```

3. Bandingkan kedua metode tersebut

Aspek	Lock Ordering	Try-Lock dengan Retry
Prinsip kerja	Semua thread mengambil kunci dalam urutan yang sama	Thread mencoba mengunci dan mengulang jika gagal
Kelebihan	Sederhana dan efisien	Fleksibel, dapat menangani situasi kompleks
Kekurangan	Butuh desain urutan yang konsisten di seluruh kode	Potensi busy waiting (loop terus mencoba)
Overhead CPU	Rendah	Lebih tinggi karena retry loop
Kapan digunakan	Jika jumlah resource dan urutannya tetap	Jika urutan resource dinamis dan sulit diprediksi

Tugas 3: Banker's Algorithm

Buat program baru dengan data:

4 proses

3 jenis resource

Available: [5, 4, 3]

Implementasikan request resource dan cek keamanan sistem

CODE

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define P 4 // jumlah proses
```

```
#define R 3 // jumlah resource
```

```
// Data awal sesuai soal
```

```
int Available[R] = {5, 4, 3};
```

```
int Max[P][R] = {
```

```
    {3, 2, 2},
```

```
    {6, 1, 3},
```

```
    {3, 1, 4},
```

```
    {4, 2, 2}
```

```
};
```

```
int Allocation[P][R] = {
```

```
    {1, 0, 0},
```

```
    {5, 1, 1},
```

```
    {2, 1, 1},
```

```
    {0, 0, 2}
```

```
};
```

```
int Need[P][R];
```

```
// -----
```

```
// Fungsi untuk menghitung Need = Max - Allocation
```

```
// -----
```

```
void calculateNeed() {
```

```
    for (int i = 0; i < P; i++) {
```

```
        for (int j = 0; j < R; j++) {
```

```
            Need[i][j] = Max[i][j] - Allocation[i][j];
```

```
        }
```

```
    }
```

```
}
```

```
// -----
```

```
// Mengecek apakah sistem dalam kondisi aman (safe state)
```

```
// -----
```

```
bool isSafe() {
```

```

int work[R];

bool finish[P] = {false};

// work = Available (copy)
for (int i = 0; i < R; i++)

    work[i] = Available[i];

int count = 0;

while (count < P) {

    bool found = false;

    for (int i = 0; i < P; i++) {

        if (!finish[i]) {

            int j;

            // Apakah Need <= Work?

            for (j = 0; j < R; j++) {

                if (Need[i][j] > work[j])

                    break;

            }

            // Jika semua Need terpenuhi

            if (j == R) {

```

```

        // Tambah kerja ke work

        for (int k = 0; k < R; k++)

            work[k] += Allocation[i][k];

        finish[i] = true;

        count++;

        found = true;

    }

}

// Tidak ada proses yang dapat dijalankan → unsafe

if (!found)

    return false;

}

return true;

}

// -----

// Fungsi Request resource

// -----

bool requestResources(int pid, int request[]) {

    printf("\nProcess %d meminta: [", pid);

```

```
for (int i = 0; i < R; i++) printf("%d ", request[i]);  
  
printf("\n");
```

```
// Cek request <= Need
```

```
for (int i = 0; i < R; i++) {  
  
    if (request[i] > Need[pid][i]) {  
  
        printf("ERROR: Request lebih besar daripada Need!\n");  
  
        return false;  
  
    }  
  
}
```

```
// Cek request <= Available
```

```
for (int i = 0; i < R; i++) {  
  
    if (request[i] > Available[i]) {  
  
        printf("Resource tidak cukup. Process harus menunggu.\n");  
  
        return false;  
  
    }  
  
}
```

```
// Alokasikan sementara
```

```
for (int i = 0; i < R; i++) {  
  
    Available[i] -= request[i];  
  
    Allocation[pid][i] += request[i];  
  
    Need[pid][i] -= request[i];  
  
}
```



```
}
```

```
// Cek apakah masih safe
```

```
if (isSafe()) {
```

```
    printf("Request DISETUJUI. Sistem aman.\n");
```

```
    return true;
```

```
}
```

```
// Rollback jika unsafe
```

```
for (int i = 0; i < R; i++) {
```

```
    Available[i] += request[i];
```

```
    Allocation[pid][i] -= request[i];
```

```
    Need[pid][i] += request[i];
```

```
}
```

```
printf("Request DITOLAK karena menyebabkan unsafe state.\n");
```

```
return false;
```

```
}
```

```
// -----
```

```
// Main Program
```

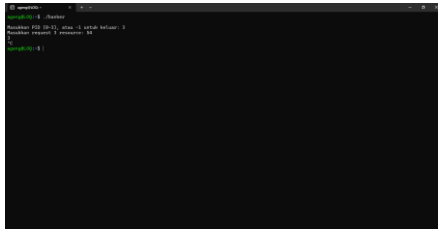
```
// -----
```

```
int main() {
```

```
    calculateNeed();
```

```
while (1) {  
  
    int pid;  
  
    printf("\nMasukkan PID (0-3), atau -1 untuk keluar: ");  
  
    scanf("%d", &pid);  
  
    if (pid == -1)  
        break;  
  
    int request[R];  
  
    printf("Masukkan request 3 resource: ");  
  
    for (int i = 0; i < R; i++)  
        scanf("%d", &request[i]);  
  
    requestResources(pid, request);  
  
    printf("Available sekarang: [");  
  
    for (int i = 0; i < R; i++) printf("%d ", Available[i]);  
  
    printf("]\n");  
}  
  
return 0;  
}
```

Contoh output



Tugas 4: Kasus Nyata (Real Case)

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

Orang A: transfer dari Rekening 1 ke Rekening 2

Orang B: transfer dari Rekening 2 ke Rekening 1

Implementasikan deadlock prevention

CODE

VERSI 1: Program Deadlock (Tanpa Pencegahan)

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* transferA(void* arg) {
```

```
    printf("A: Mengunci Rekening 1...\n");
```

```
    pthread_mutex_lock(&rekening1);
```

```
sleep(1);

printf("A: Menunggu Rekening 2...\n");

pthread_mutex_lock(&rekening2);

printf("A: Transfer 1 -> 2 selesai\n");

pthread_mutex_unlock(&rekening2);

pthread_mutex_unlock(&rekening1);

return NULL;
}
```

```
void* transferB(void* arg) {

    printf("B: Mengunci Rekening 2...\n");

    pthread_mutex_lock(&rekening2);

    sleep(1);

    printf("B: Menunggu Rekening 1...\n");

    pthread_mutex_lock(&rekening1);

    printf("B: Transfer 2 -> 1 selesai\n");

    pthread_mutex_unlock(&rekening1);

    pthread_mutex_unlock(&rekening2);
```

```

        return NULL;
    }

int main() {
    pthread_t tA, tB;

    pthread_create(&tA, NULL, transferA, NULL);
    pthread_create(&tB, NULL, transferB, NULL);

    pthread_join(tA, NULL);
    pthread_join(tB, NULL);

    return 0;
}

```

VERSI 2: Deadlock Prevention (Solusi)

```

#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;

// Fungsi transfer AMAN TANPA DEADLOCK

```

```
void lock_ordered(pthread_mutex_t* first, pthread_mutex_t* second) {  
  
    pthread_mutex_lock(first);  
  
    pthread_mutex_lock(second);  
  
}
```

```
void unlock_ordered(pthread_mutex_t* first, pthread_mutex_t* second) {  
  
    pthread_mutex_unlock(second);  
  
    pthread_mutex_unlock(first);  
  
}
```

```
void* transferA(void* arg) {  
  
    printf("A: Transfer 1 -> 2\n");  
  
    lock_ordered(&rekening1, &rekening2);  
  
    printf("A: Transfer selesai\n");  
  
    unlock_ordered(&rekening1, &rekening2);  
  
    return NULL;  
  
}
```

```
void* transferB(void* arg) {  
  
    printf("B: Transfer 2 -> 1\n");  
  
    // B tetap harus mengikuti urutan: 1 → 2  
  
    lock_ordered(&rekening1, &rekening2);  
  
    printf("B: Transfer selesai\n");  
  
    unlock_ordered(&rekening1, &rekening2);  
  
}
```

```
        return NULL;
    }

int main() {

    pthread_t tA, tB;

    pthread_create(&tA, NULL, transferA, NULL);

    pthread_create(&tB, NULL, transferB, NULL);

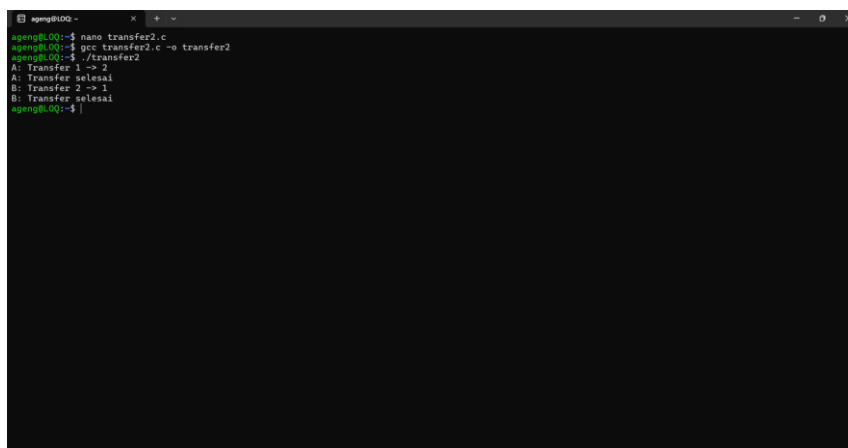
    pthread_join(tA, NULL);

    pthread_join(tB, NULL);

    return 0;

}
```

Output program

A terminal window titled 'ageng@LUG2-' with standard window controls. The terminal shows the following commands and output:

```
ageng@LUG2:~$ nano transfer2.c
ageng@LUG2:~$ gcc transfer2.c -o transfer2
ageng@LUG2:~$ ./transfer2
A: Transfer 1 -> 2
A: Transfer selesai
B: Transfer 2 -> 1
B: Transfer selesai
ageng@LUG2:~$
```